

人工智能数学原理与算法 B

第三次实验

逻辑回归与大学排名分类

分阶段实验手册

主讲 胡泷

整理 胡泷、曾有成

任课教师 钟志诚

单位 中国科学技术大学
人工智能与数据科学学院

从数据到可解释的分类流程

数据清洗与划分 → 数值表示 → 线性得分与概率 → 交叉熵与正则化 → 梯度更新 → 独立评估。

2026 年秋季 建议 3 学时 (135 分钟)

C0-C7 分阶段完成；日期与提交平台以课程通知为准。

随包数据为 QS (Quacquarelli Symonds) 2026 排名的教学整理版。

目录

1	实验目的与实验环境	1
2	实验安排与验收方式	3
3	C0–C1：数据观察与清洗	4
4	C2：数值与类别特征表示	6
5	C3–C4：从样本到模型	8
6	C5–C6：训练目标与一次更新	10
7	C7：模型选择、解释与最终测试	12
8	选做：特征选择对照	13
9	理论与代码的对应关系	14
10	实验报告要求与提交	18
11	资料与改编说明	20

1 实验目的与实验环境

1.1 实验目的

1. 理解逻辑回归将线性得分经 sigmoid 函数转换为概率、再按阈值给出类别的过程，区分得分、概率与预测类别三种输出。
2. 掌握混合类型表格数据的预处理方法，包括数值清洗与均值填补、仅由训练集拟合的标准化与独热编码，以及缺失和未见类别的处理。
3. 实现二元交叉熵损失与 L1/L2 正则化，手算并用程序核对一次梯度更新，完成小批量随机梯度下降训练。
4. 按训练集、验证集与测试集的职责完成模型选择与最终评价，并结合混淆矩阵、多数类基线与多项指标解释分类结果。

1.2 实验任务

本实验使用 QS (Quacquarelli Symonds) 2026 世界大学排名的教学整理数据，依据学校的三个已公布指标分数与五个类别属性，判断该校是否位于同年榜单前 500 名。该任务是同年榜单内的分类练习，既不预测未来排名，也不对未上榜的大学作出评价。

课前应阅读《前置讲义》第 1-5 节，复习表格读取、缺失填补、独热编码与批次划分，预计用时约 30 分钟。课堂部分通过数学推导、手算与检查点完成模型的训练与评价，建议用时 3 学时（135 分钟），环境安装时间不计在内。

1.3 实验环境与文件

本实验沿用实验 01 配置的 conda 环境。进入 `code/` 目录后执行：

```
conda activate ustc-ai-experiments-26fall-B
python checkpoints.py C0
```

若提示缺少依赖，可运行 `python -m pip install -r requirements.txt` 安装。参考环境为 Python 3.11、PyTorch 2.14、pandas 3.0 与 NumPy 2.4。程序在 CPU (Central Processing Unit, 中央处理器) 上即可完成运行，不需要加速硬件，也不依赖 scikit-learn。实验所用文件及其作用如下表所示。

文件	作用
<code>student.py</code>	学生主要补全的文件，共 17 处 TODO，每处均附 Hint 与可核对的预期结果
<code>checkpoints.py</code>	C0-C7 各阶段的小样例自检，可按阶段单独运行
<code>support.py</code> , <code>workflow.py</code>	已提供的预处理编排、模型选择与结果输出代码
<code>run_lab.py</code> , <code>logic.ipynb</code>	脚本与 Notebook 两种运行入口，二者调用同一份学生实现
<code>data/classroom/</code>	固定的课堂数据、来源与划分记录
<code>report-template.md</code>	实验报告模板

1.4 实验方法与注意事项

各检查点均按以下步骤进行：先运行已给出的小例子并观察输出，修改一个数值后预测结果的变化；再根据本手册给出的函数规格补全对应函数，运行自检，并解释所得结果。每个检查点的规格框列出了各函数的输入（参数名、类型与形状）、输出与提示，与 `student.py` 中的文档字符串一致。

Notebook 通过普通的 `import student` 调用同一份代码，因此修改 `student.py` 后需重启内核，避免在两处分别维护实现。骨架中的 `return None` 仅为待完成的占位，并非算法的正确输出。

明确模型的输入边界

模型的输入仅包括三个数值分数与五个类别字段；学校名称、原表行号、各类排名、总分与历史排名均不得作为输入。类别缺失值统一编码为 `Unknown`，数值缺失值以训练集均值填补；填补规则与字段取舍均不得依据测试集确定。

2 实验安排与验收方式

2.1 课堂时间安排

课堂按检查点分为八个阶段，各阶段的教师活动、学生活动与产出如下表所示。

阶段/分钟	教师活动	学生活动与产出
C0 / 10	由决策树的概率输出引入本实验，演示张量形状	运行矩阵乘法样例，记录修改权重后的输出
C1 / 15	区分标签、标识字段与数值缺失	补全两个清洗函数，记录字段类型与缺失统计
C2 / 25	手算标准化与独热编码，说明未知类别的处理	补全五个函数，核对列顺序与张量形状
C3 / 10	说明样本配对与末批处理	补全数据集类与加载器，用五条样本自检
C4 / 15	联系得分、概率与预测类别	补全参数注册与前向计算，核对得分 5
C5 / 20	推导交叉熵与权重惩罚	补全损失与惩罚项，验证截距不受惩罚
C6 / 20	手算一次更新，讲解五步训练循环	补全训练函数，核对连续两次更新
C7 / 20	区分模型选择与最终测试，讨论错误代价	补全指标函数，比较方案并在冻结后测试

各阶段合计 $10 + 15 + 25 + 10 + 15 + 20 + 20 + 20 = 135$ 分钟。完整训练运行期间可同步分析输出图表；特征选择对照安排为课后选做内容，不占用必做部分的课堂时间。

2.2 阶段依赖与验收证据

C0 可独立运行；C1 完成数值与标签清洗；C2 的数值与类别小测试不依赖真实数据；C3 使用自检提供的张量；C4 完成模型定义。C5 依赖 C4，C6 依赖 C3–C5；C7 的指标小例子可独立检查，完整实验则需要全部阶段完成。

检查点验收

每个阶段需在报告中记录对应函数、实现思路、一条小例子的输入与预期/实际输出，以及对结果的解释。**PASS** 仅表明所列样例通过，不能替代报告中的分析；未完成的内容应如实说明。

2.3 必做与选做内容

必做内容包括 C0–C7、一次梯度手算、三种正则化方案的比较、模型冻结后的测试以及完整的实验报告。选做内容为仅在训练集和验证集上比较完整与精简两组特征。两部分均不以准确率高低作为评分门槛。

板书安排

左栏：数据角色与编码表；中栏： $\mathbf{X}\boldsymbol{\theta} \rightarrow \mathbf{p} \rightarrow R \rightarrow \nabla R$ （设计矩阵、参数、概率、平均损失、梯度）；右栏：训练/验证/测试的职责与四种分类计数。详细的维度与记号见前置讲义。

3 C0–C1: 数据观察与清洗

3.1 C0: 运行形状小例子

运行 `python checkpoints.py C0`。程序应输出训练原表形状 (902,11)、缺失标签数 0, 以及矩阵乘法结果 `[[2.0], [-1.0]]`。这 11 列包含学校名称、原表行号与标签, 因此不能理解为 11 个模型输入。修改权重后, 应先手算预期结果, 再运行程序核对。

C0 输入、输出与提示

本阶段无需编写函数。

输入: `X = [[1, 2], [1, -1]]` (形状 (2,2) 的 `torch.Tensor`) 与 `w = [[0], [1]]` (形状 (2,1))。

输出: `X @ w = [[2.0], [-1.0]]`, 形状 (2,1); 同时打印训练表形状 (902, 11) 与缺失标签数 0。

提示: 矩阵乘法结果的行数等于 `X` 的行数、列数等于 `w` 的列数; 例如将 `w` 改为 `[[1], [1]]`, 结果应为 `[[3.0], [0.0]]`。

3.2 C1: 补全标签过滤与数值清洗函数

在 `student.py` 中补全 `filter_labels` 与 `clean_numeric`。前者仅删除标签缺失的行, 不填补任何特征; 后者将数值字符串转换为数值, 空白、横线和无法解析的文本转换为缺失值, 而不是填为 0。可用以下代码查看训练表的形状、缺失统计与类别分布:

```
import support
train = support.load_split(support.DATA_DIR, "train")
print(train.shape)
print(train.isna().sum())
print(train["top500"].value_counts())
print(train["status"].value_counts(dropna=False))
```

C1 输入、输出与提示

`filter_labels(frame)`

输入: `frame (pd.DataFrame)`: N 行的表格, 必须含 `top500` 列, 取值为 0、1 或 `NaN`。

输出: `pd.DataFrame`: 只保留标签非缺失行的新表, 行索引从 0 重新编号; 列与输入相同, 特征缺失保持不变。

提示: `dropna(subset=["top500"])` 只检查标签列, 随后调用 `reset_index(drop=True)`。

`clean_numeric(series)`

输入: `series (pd.Series)`: 长度为 N 的一列, 元素可为 `str`、`float`、`int` 或 `None`。

输出: `pd.Series`: 长度为 N 的数值列, 无法解析的值为 `NaN`, 数值 0 保留为 0。

提示: 先将 `"-"` 与 `" "` 替换为 `None`, 再调用 `pd.to_numeric(..., errors="coerce")`。

自检: 标签为 1、缺失、0 的三条记录过滤后保留两条, 标签为 `[1, 0]`, 且第一条记录的特征缺失保持不变; `["12.5", "-", "", None]` 转换为 `[12.5, NaN, NaN, NaN]`; `"0"` 转换为 `0.0`。

3.3 输入字段与标签说明

课堂数据中的 `region`、`status`、`size`、`focus`、`research` 为类别字段，三个以 `score` 结尾的字段为指标分数。`name` 与 `row_id` 仅用于追溯数据来源，`top500` 为标签。标签依据源表的实际名次及并列关系生成，正类共 502 所，并非按表中前 500 行截取。

训练集、验证集与测试集的样本数与正类数分别为 902/301、300/100、302/101。数据中保留了真实缺失，无需人为制造缺失。数据构造脚本由教师提供，学生无需重写；原始镜像与构造规则见数据说明。

报告记录要求

列出八个输入字段及其类型，说明原表行号也必须排除的原因，并记录训练集的缺失统计。标签缺失的处理以小例子验证即可，不得在真实数据中伪造标签缺失。

4 C2: 数值与类别特征表示

4.1 数值分支: 三个函数

补全 `fit_statistics`、`apply_statistics` 与 `to_tensors`。具体步骤为: 先计算训练集各列非缺失值的均值并用其填补缺失, 再计算填补后的总体标准差; 对验证集和测试集进行转换时复用训练统计量, 不重新计算。最后将特征转换为浮点张量, 在首列增加常数 1, 并将标签整理为列向量。

C2 数值分支的输入、输出与提示

`fit_statistics(features)`

输入: `features` (`pd.DataFrame`): $N \times k$ 的数值训练特征, 元素为 `float` 或 `NaN`, 不含标签、编号和常数列。

输出: 元组 (`means`, `scales`), 两者均为长度 k 、以列名为索引的 `pd.Series`。

提示: `mean()` 自动跳过 `NaN`; 全空列的均值记为 0; 用 `std(ddof=0)` 计算总体标准差, 值为 0 的尺度改为 1。

`apply_statistics(features, means, scales)`

输入: `features` (`pd.DataFrame`): $M \times k$ 的数值特征, 列名与训练集一致; `means`、`scales` (`pd.Series`): 长度为 k 的训练统计量。

输出: `pd.DataFrame`: $M \times k$ 的标准化特征, 不含 `NaN`, 行索引与列名不变。

提示: 先 `fillna(means)`, 再减去 `means`、除以 `scales`; `pandas` 按列名自动对齐。

`to_tensors(features, labels)`

输入: `features` (`pd.DataFrame`): $N \times d$ 的已处理特征; `labels` (`pd.Series`): 长度为 N 的 0/1 标签。

输出: 元组 (`X`, `y`): `X` 为形状 $(N, d+1)$ 的 `torch.float32` 张量, 首列全为 1; `y` 为形状 $(N, 1)$ 的 `torch.float32` 张量。

提示: `torch.ones((N, 1))` 与特征张量用 `torch.cat(..., dim=1)` 按列拼接; 标签用 `reshape(-1, 1)`。

数值核对

训练值为 `[1, 3, 缺失]` 时, 均值为 2, 填补后为 `[1, 3, 2]`, 总体标准差约为 0.816497; 验证值 100 标准化后约为 120.025, 验证集中的缺失值标准化后为 0。训练均值不会因验证集中出现较大的值而改变。此外, 常数列的尺度约定为 1, 全部缺失的训练列均值约定为 0。

4.2 类别分支: 两个函数

补全 `fit_categories` 与 `encode_categories`。前者对每个类别字段收集训练集中出现的非缺失取值, 排序后在末尾追加 `Unknown`; 后者按固定的类别表逐列生成 0/1 指示变量, 缺失值与训练集中未出现的类别统一归入 `Unknown`。

C2 类别分支的输入、输出与提示

`fit_categories(frame)`

输入: `frame` (`pd.DataFrame`): N 行、只含类别字段的训练表, 元素为 `str` 或 `NaN`。

输出: `dict`: 键为字段名 (`str`), 值为按字母排序的类别列表 (`list of str`), 最后一项固定为 `"Unknown"`。

提示: 逐列调用 `dropna().unique()` 后用 `sorted()` 排序; 类别表只能由训练集拟合。

encode_categories(frame, categories)

输入: frame (pd.DataFrame): M 行的待编码表; categories (dict): fit_categories 的返回值。

输出: pd.DataFrame: M 行、元素为 0.0 或 1.0 的浮点表, 行索引与 frame 相同, 列名形如 region=Asia。

提示: 先 fillna("Unknown"), 再把不在类别表中的取值替换为 "Unknown"; 每个类别列为 (values == level).astype(float)。

自检: 训练取值为 Asia、Europe 和缺失时, 类别表为 {"Region": ["Asia", "Europe", "Unknown"]}; 验证取值 Europe、Atlantis、缺失的编码依次为 (0,1,0)、(0,0,1)、(0,0,1)。其中 Atlantis 是为检查未知类别处理而虚构的取值, 并非源数据中的地区。

可用以下代码查看真实数据的处理结果:

```
import student
train, val, state = support.prepare_train_val(
    student, support.DATA_DIR)
print(train[0].shape, train[1].shape)
print(val[0].shape, val[1].shape)
print(state["categories"])
```

运行 python checkpoints.py C2。完整特征由 3 个标准化数值列与 25 个独热列组成; 训练集输出形状为 (902,29) 与 (902,1), 验证集为 (300,29) 与 (300,1)。29 列中包含新增的首列常数; 此时测试文件尚未被读取。

报告记录要求

分别给出数值填补与独热编码的一条证据, 并解释未知类别的处理、列顺序与首列常数的作用。不得在验证集或测试集上单独拟合类别表, 也不应对标签或独热列进行标准化。

5 C3–C4: 从样本到模型

5.1 C3: 保持特征与标签成对

补全 `UniversityDataset` 的 `__len__`、`__getitem__` 与 `make_loader`。构造函数已保存 `X` 与 `y`，每个索引应返回对应的一对张量。仅在训练时打乱样本顺序，并保留不足一个批量的最后一批。

C3 输入、输出与提示

`UniversityDataset(X, y)`

输入: `X` (`torch.Tensor`): 形状 $(N, d+1)$ 的增广特征; `y` (`torch.Tensor`): 形状 $(N, 1)$ 的标签。构造函数已提供。

输出: `__len__()` 返回 `int` 类型的样本数 N ; `__getitem__(index)` 接收 `int` 索引 (0 至 $N-1$)，返回元组 $(X[index], y[index])$ ，形状为 $(d+1,)$ 与 $(1,)$ 。

提示: `len(self.X)` 即为行数; 两个张量必须使用同一个索引，不能分别打乱。

`make_loader(dataset, batch_size, training)`

输入: `dataset` (`UniversityDataset`); `batch_size` (`int`)，如 64; `training` (`bool`)，`True` 表示训练。

输出: `torch.utils.data.DataLoader`: 每次迭代给出 (X_batch, y_batch) ，形状为 $(B, d+1)$ 与 $(B, 1)$ ，最后一批允许 B 小于 `batch_size`。

提示: 设置 `shuffle=training`、`drop_last=False`、`num_workers=0`。

自检: 5 条样本、每批 2 条且不打乱时，批次大小依次为 2、2、1，拼接后与原数据一致; 开启打乱后，每批特征与标签仍保持配对。

```
dataset = student.UniversityDataset(*train)
loader = student.make_loader(dataset, 64, True)
for X_batch, y_batch in loader:
    print(X_batch.shape, y_batch.shape)
```

运行 `python checkpoints.py C3`。真实训练集 902 条、每批 64 条时共 15 批，末批为 6 条。

5.2 C4: 线性得分与概率

补全 `sigmoid` 以及模型类的 `__init__` 与 `forward`。模型参数为一个列向量，其行数等于增广后的特征维数，需用 `nn.Parameter` 注册并初始化为零。`forward` 返回矩阵乘积得到的线性得分，`sigmoid` 仅在计算概率时单独调用。

C4 输入、输出与提示

`sigmoid(z)`

输入: `z` (`torch.Tensor`): 任意形状的线性得分。

输出: `torch.Tensor`: 与 `z` 形状相同的概率，取值在 0 与 1 之间。

提示: 公式为 $1/(1+e^{-z})$ ，实现时调用 `torch.sigmoid(z)` 以避免极端值溢出。

`LogisticRegression(input_dim)`

输入: `input_dim` (`int`): 增广后的特征数 $d+1$ ，主实验中为 29。

输出: 无返回值; 注册形状为 $(input_dim, 1)$ 的参数 `self.theta`。

提示: `nn.Parameter(torch.zeros((input_dim, 1)))`; 只有注册为 `Parameter` 的张量才会被优化器更新。

forward(X)

输入: `X` (`torch.Tensor`): 形状 $(B, d+1)$ 的 `float32` 增广特征。

输出: `torch.Tensor`: 形状 $(B, 1)$ 的线性得分 (logits), 尚未经过 `sigmoid`。

提示: 使用矩阵乘法 `@`, 不要使用逐元素乘法 `*`。

自检: $z = -2, 0, 2$ 时概率约为 0.119203、0.5、0.880797; 参数为 $(1, 2, -1)^\top$ 、增广输入为 $(1, 3, 2)^\top$ 时, `forward` 返回得分 5, 对应概率约为 0.993307。

```
model = student.LogisticRegression(train[0].shape[1])
logits = model(train[0][:3])
probabilities = student.sigmoid(logits)
print(logits.shape, probabilities)
```

运行 `python checkpoints.py C4`。注意 `forward` 应返回得分而非概率。

报告记录要求

C3 部分记录样本配对与末批处理的结果; C4 部分记录输入、参数、得分、概率与张量形状, 并说明矩阵乘法 `@` 与逐元素乘法 `*` 的区别。

6 C5–C6: 训练目标与一次更新

6.1 C5: 二元交叉熵与权重惩罚

补全 `data_loss` 与 `penalty`。数据损失采用二元交叉熵 BCE (Binary Cross-Entropy)，由 `BCEWithLogitsLoss` 直接接收原始得分，并以数值稳定的方式计算损失。

L1 (L1 Norm, 一范数) 惩罚为特征权重绝对值之和；L2 (L2 Norm, 二范数) 惩罚为权重二范数平方的一半。两种惩罚均只作用于 `theta[1:]`，截距不受惩罚。

C5 输入、输出与提示

`data_loss(logits, y)`

输入: `logits` (`torch.Tensor`): 形状 $(B, 1)$ 的原始得分；`y` (`torch.Tensor`): 形状 $(B, 1)$ 的 `float32` 0/1 标签。

输出: `torch.Tensor`: 0 维标量张量，为本批平均二元交叉熵，保留计算图。

提示: 使用 `nn.BCEWithLogitsLoss()`，默认 `reduction="mean"`；不要先调用 `sigmoid`，也不要调用 `item()`。

`penalty(kind)`

输入: `kind` (`str`): 正则化类型，取 `"none"`、`"l1"` 或 `"l2"`。

输出: `torch.Tensor`: 可反向传播的 0 维标量张量。

提示: 先取 `w = self.theta[1:]`；L1 为 `w.abs().sum()`，L2 为 `0.5 * (w ** 2).sum()`，`"none"` 可返回 `self.theta.sum() * 0.0`。

自检: 得分全为 0 时平均交叉熵为 $\log 2 \approx 0.693147$ ，得分为 ± 1000 的极端值仍为有限数；参数为 $(10, 3, -4)$ 时 L1 与 L2 惩罚分别为 7 和 12.5，L2 惩罚对截距的梯度为 0。

运行 `python checkpoints.py C5` 核对上述结果。

6.2 C6: 补全训练循环

补全 `train_epoch`。每个批次依次执行五个步骤：清除梯度、前向计算、构造“数据损失加正则惩罚”的总目标、反向传播、更新参数；随后累计本批的数据损失，用于计算返回值。SGD (Stochastic Gradient Descent, 随机梯度下降) 的自动微分与参数更新器已经提供，学生只需按顺序连接上述步骤。

```
optimizer.zero_grad()
logits = model(X)
loss = data_loss(logits, y)
objective = loss + strength * model.penalty(kind)
objective.backward()
optimizer.step()
total += loss.detach().item() * len(y)
count += len(y)
```

训练过程中必须保留计算图，反向传播之前不能对损失调用 `item()` 或 `detach()`。每个批次都需清除梯度。本函数的返回值是各批更新前的数据损失；绘图使用的则是每轮结束后在完整数据上重新计算的数据损失。

C6 输入、输出与提示

`train_epoch(model, loader, optimizer, kind, strength)`

输入: `model` (`LogisticRegression`); `loader` (`DataLoader`)，每次给出形状

$(B, d+1)$ 与 $(B, 1)$ 的 (X, y) ; `optimizer (torch.optim.Optimizer)`, 如 `torch.optim.SGD`; `kind (str)`, 取 "none"、"l1" 或 "l2"; `strength (float)`, 正则化系数 λ , 如 0.01。

输出: `float`: 本轮各批更新前数据损失按样本数加权的平均值, 不含正则项; 同时原地更新 `model.theta`。

提示: 按上方代码的顺序执行; 记录损失时使用 `loss.detach().item()`, 并累加 `total` 与 `count`, 否则函数返回 `None`。

自检: 使用下方手算例题的两条样本、学习率 0.4 训练一轮, 参数应变为 $(0, 0.1, -0.1)^T$; 再训练一轮, 结果应与独立计算的第二次更新一致。

手算后再运行

两所假设学校的增广输入分别为 $(1, 1, 0)$ 与 $(1, 0, 1)$, 标签分别为 1 与 0。参数为零时两者的概率均为 0.5, 平均梯度为 $(0, -0.25, 0.25)^T$ 。取学习率 0.4、不加正则时, 更新后的参数为 $(0, 0.1, -0.1)^T$ 。推导过程见前置讲义第 8 节。

运行 `python checkpoints.py C6`。自检还会核对第二次更新的结果, 用于发现遗漏清除梯度的问题。

报告记录要求

C5 部分记录损失及两种惩罚的数值; C6 部分记录一次手算的输入、概率、梯度与更新结果, 并与程序输出对照。仅给出“损失下降”不足以证明更新正确。

7 C7: 模型选择、解释与最终测试

7.1 补全评价指标函数

补全 `binary_metrics`。以前 500 名为正类, TP (True Positive, 真正例)、FP (False Positive, 假正例)、FN (False Negative, 假负例) 与 TN (True Negative, 真负例) 分别统计四种预测结果。分母为零时, 相应指标按 0 处理。

C7 输入、输出与提示

`binary_metrics(y, probabilities, threshold=0.5)`

输入: `y` (`torch.Tensor`): 形状 $(N, 1)$ 的真实标签, 元素为 0.0 或 1.0; `probabilities` (`torch.Tensor`): 形状 $(N, 1)$ 的预测概率; `threshold` (`float`): 决策阈值, 默认 0.5。

输出: `dict`: 键 `tn`、`fp`、`fn`、`tp` 对应 `int` 计数, 键 `accuracy`、`precision`、`recall`、`f1` 对应 `float` 指标。

提示: 先用 `reshape(-1)` 转为一维; 预测类别为 `probabilities >= threshold`, 恰好等于阈值时判为正类; 用布尔运算 `&` 与 `~` 统计四种情况, 不能直接比较概率与标签。

自检: 标签 `[1, 0, 1, 0]`、概率 `[0.9, 0.8, 0.4, 0.1]` 在阈值 0.5 下四种计数均为 1, 四项指标均为 0.5; 阈值为 1.0 时没有预测正类, 精确率记为 0。

7.2 在验证集上选择模型

运行小样例与全部自检后, 执行完整训练:

```
python checkpoints.py all
python run_lab.py
```

默认输出目录为 `code/outputs-qs/`。训练采用零初始化、训练种子 42、批量大小 64、学习率 0.05, 最多训练 60 轮, 比较无正则、L2 系数 0.01 与 L1 系数 0.01 三种方案。每种方案取验证数据损失最低的一轮作为该方案的结果, 再从三种方案中选择验证损失最低者; 决策阈值预先固定为 0.5。程序输出以下文件:

- `candidates.csv`: 三种方案的设置与验证结果。
- `none_loss.png`、`l2_loss.png`、`l1_loss.png`: 同一组参数下的训练与验证数据损失曲线, 不包含惩罚项。
- `selection.json`、`preprocessing.json`、`selected.pt`: 冻结的方案、训练集预处理状态与模型权重。
- `validation_thresholds.csv`: 阈值为 0.3、0.5、0.7 时的验证集计数与指标, 仅用于讨论阈值的权衡。
- `weights.csv`: 各特征名对应的参数值。在本实验的更新方式下, L1 正则不保证产生精确为零的权重。

7.3 记录选择后进行最终测试

在报告中记录冻结方案后, 执行:

```
python run_lab.py --final-test
```

程序恢复冻结的模型, 仅使用保存的训练均值、尺度与类别表处理测试集, 并输出 `test_metrics.json` 与 `test_predictions.csv`。多数类基线将全部测试样本预测为训练集中的多数类, 即负类。报告中应给出准确率、精确率、召回率、F1 分数 (F1 Score) 与混淆矩阵; 混淆矩阵的行表示

真实类别，列表示预测类别。

报告记录要求

记录上述四条小例子在阈值 0.5 下的计数，并解释阈值降低后计数的变化。综合实验部分需记录所选方案、一张损失曲线和混淆矩阵，不能仅凭准确率得出结论。

8 选做：特征选择对照

8.1 实验设计

在查看测试结果之前，可执行以下命令进行特征选择对照：

```
python run_lab.py --feature-study
```

完整组使用三个数值字段与五个类别字段；精简组仅删除 focus 与 research 两个字段。两组使用相同的数据划分，并固定 L2 系数 0.01、学习率 0.05、批量大小 64、训练种子 42 与最大轮数 60，各组均取验证损失最低的一轮。特征维数由 28 降为 18，增广后分别为 29 和 19。

结果保存在 `outputs-qs/feature_study/comparison.csv`。选做实验不读取测试集，不改变主实验的冻结模型，也不得反复使用测试结果进行字段选择。该对照检验的是整组字段的作用，并不能逐一证明每个字段都重要。

8.2 结果解释

报告中应列出保留与删除的字段、特征维数、验证损失、准确率、精确率、召回率与 F1 分数。若精简组表现变差，说明被删除的字段提供了有用信息；若两组表现相近，可讨论模型复杂度与可解释性之间的取舍。不应得出“特征越少越好”或“正则化一定提高准确率”之类的结论。

两个常见混淆

删除泄漏字段是建模前必须完成的筛选，不能当作可自由调节的性能技巧。L1 惩罚鼓励稀疏，但普通梯度更新不保证得到精确为零的权重，因此不能把较小的权重直接视为对应字段已被删除。

8.3 课堂讨论问题与参考答案

1. 为什么不把学校规模直接编码为 1、2、3、4？

参考答案：这样会隐含相邻档位作用等距的假设，独热编码不作此假设。

2. 为什么不按全部数据收集类别？

参考答案：编码表同样属于从训练数据中学习的内容，验证集与测试集中的未知类别应按预先约定的规则处理。

3. 两个方案的准确率相同，能否认为二者完全等价？

参考答案：不能。二者的概率损失和四种计数都可能不同。

4. 为什么不直接使用总分作为输入？

参考答案：本任务的标签由排名构造，总分几乎直接提供了标签信息，会造成泄漏。

9 理论与代码的对应关系

C0–C7 与选做部分按程序的执行顺序展开, 本节则按逻辑回归的理论主线回顾各个公式在代码中的实现位置, 便于检查“写出的代码是否就是推导出的算法”。本节用于课后复习, 不占用 135 分钟的课堂时间; 撰写报告的“实验原理”部分时, 可按本节的顺序组织内容。完整推导见前置讲义第 3–9 节, 这里只列出关键结论及其代码对应。

本节沿用前置讲义的记号: N 为样本数, B 为一个批次的样本数, $d = 28$ 为编码后的特征数; 括号上标 (n) 表示样本编号, 方括号上标 $[t]$ 表示迭代次数。样本按行排列, 因此代码中张量的第一维始终是样本维。

9.1 从原始字段到增广设计矩阵

对第 j 个数值特征, 先用训练集均值 μ_j 填补缺失得到 $\bar{x}_j^{(n)}$, 再用训练集总体标准差 s_j 标准化:

$$u_j^{(n)} = \frac{\bar{x}_j^{(n)} - \mu_j}{s_j}. \quad (9.1.1)$$

其中 μ_j 与 s_j 由 `fit_statistics` 在训练集上计算一次, `apply_statistics` 在训练、验证与测试集上复用, 二者分别对应“估计参数”和“应用变换”两个步骤。类别字段由 `fit_categories` 确定类别表, 再由 `encode_categories` 转换为 0/1 指示列。

三个标准化数值与 25 个独热列拼成处理后的特征向量 $\tilde{\mathbf{x}}^{(n)} \in \mathbb{R}^d$ 。在其前面增加常数 1 即得到增广向量, 所有样本按行堆叠为设计矩阵:

$$\tilde{\mathbf{x}}^{(n)} = (1, (\mathbf{x}^{(n)})^\top)^\top \in \mathbb{R}^{d+1}, \quad \mathbf{X} = \begin{bmatrix} (\tilde{\mathbf{x}}^{(1)})^\top \\ \vdots \\ (\tilde{\mathbf{x}}^{(N)})^\top \end{bmatrix} \in \mathbb{R}^{N \times (d+1)}. \quad (9.1.2)$$

这一步由 `to_tensors` 完成: `torch.ones((N, 1))` 就是 $\mathbf{X}$ 的第一列, 返回的 $\mathbf{X}$ 形状为 $(902, 29)$, $\mathbf{y}$ 形状为 $(902, 1)$, 分别对应 $\mathbf{X}$ 与标签列向量 $\mathbf{y}$ 。`UniversityDataset` 的第 n 个元素即 $(\tilde{\mathbf{x}}^{(n)}, y^{(n)})$, `make_loader` 每次取出 B 行组成小批量 $\mathbf{X}_B \in \mathbb{R}^{B \times (d+1)}$ 。

9.2 模型: 得分、概率与类别

参数向量 $\boldsymbol{\theta} = (\theta_0, \theta_1, \dots, \theta_d)^\top$ 在代码中是形状为 $(d+1, 1)$ 的 `self.theta`, 其中 `theta[0]` 是截距 θ_0 , 与 $\mathbf{X}$ 的常数列相乘。一个批次的线性得分、概率与预测类别为

$$\mathbf{z} = \mathbf{X}_B \boldsymbol{\theta}, \quad \mathbf{p} = \sigma(\mathbf{z}), \quad \hat{\mathbf{y}}^{(n)} = \mathbf{1}[p^{(n)} \geq \tau], \quad (9.2.1)$$

其中 $\sigma(z) = 1/(1+e^{-z})$ 逐元素作用, τ 为决策阈值, $\mathbf{1}[\cdot]$ 在条件成立时取 1。三个量分别对应 `forward` 中的 `X @ self.theta`、`sigmoid(logits)`, 以及 `binary_metrics` 中的 `probabilities >= threshold`。代码把三者放在不同函数中, 正是为了区分“得分”“概率”和“类别”这三种输出: `forward` 不调用 `sigmoid`, 损失函数直接接收得分。

9.3 目标函数: 交叉熵与正则化

由伯努利似然取负对数并对批次平均, 得到批次经验风险 (平均二元交叉熵):

$$R_B(\boldsymbol{\theta}) = -\frac{1}{B} \sum_{n=1}^B \left[y^{(n)} \log p^{(n)} + (1 - y^{(n)}) \log(1 - p^{(n)}) \right]. \quad (9.3.1)$$

`data_loss` 中的 `nn.BCEWithLogitsLoss()` 计算的正是该式: 其默认 `reduction="mean"` 对应式中的 $1/B$ 。它接收得分 z 而非概率 p , 内部采用等价的稳定形式 $\max(z, 0) - yz + \log(1 + e^{-|z|})$, 从而避免计算 $\log 0$ 或过大的指数。

训练时实际最小化的总目标为

$$J_B(\boldsymbol{\theta}) = R_B(\boldsymbol{\theta}) + \lambda \Omega(\boldsymbol{\theta}), \quad \Omega_{L1} = \sum_{j=1}^d |\theta_j|, \quad \Omega_{L2} = \frac{1}{2} \sum_{j=1}^d \theta_j^2. \quad (9.3.2)$$

求和从 $j = 1$ 开始, 即不惩罚截距, 对应 `penalty` 中的 `theta[1:]`; λ 对应参数 `strength`; 总目标对应 `train_epoch` 中的 `objective = loss + strength * model.penalty(kind)`。注意 `loss` 只是 R_B , 因此训练曲线与模型选择使用的都是不含惩罚项的数据损失。

9.4 梯度与参数更新

对单样本有 $\partial \mathcal{L}^{(n)} / \partial z^{(n)} = p^{(n)} - y^{(n)}$, 再由 $\mathbf{z} = \mathbf{X}_B \boldsymbol{\theta}$ 得到批次梯度; L2 惩罚的梯度只在特征权重上非零:

$$\nabla_{\boldsymbol{\theta}} R_B = \frac{1}{B} \mathbf{X}_B^{\top} (\mathbf{p} - \mathbf{y}_B), \quad \nabla_{\boldsymbol{\theta}} \Omega_{L2} = (0, \theta_1, \dots, \theta_d)^{\top}. \quad (9.4.1)$$

一次小批量随机梯度下降的更新为

$$\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \eta \nabla_{\boldsymbol{\theta}} J_B(\boldsymbol{\theta}^{[t]}), \quad (9.4.2)$$

其中 η 为学习率, 对应创建 `torch.optim.SGD` 时传入的 `lr=0.05`。代码中并不手写梯度公式, 三个调用分别承担以下职责:

- `objective.backward()`: 按链式法则自动计算 $\nabla_{\boldsymbol{\theta}} J_B$, 结果存入 `model.theta.grad`;
- `optimizer.step()`: 执行式中的参数更新;
- `optimizer.zero_grad()`: 在每个批次开始时清除旧梯度。PyTorch 默认把新梯度累加到 `.grad` 上, 若不清除, 实际使用的将是多个批次梯度之和。

下面的代码用 C6 手算例题中一次更新后的参数 $(0, 0.1, -0.1)^{\top}$, 核对自动微分结果与公式是否一致 (需先完成 C4 与 C5):

```
import torch
import student
X = torch.tensor([[1., 1., 0.], [1., 0., 1.]])
y = torch.tensor([[1.], [0.]])
model = student.LogisticRegression(3)
with torch.no_grad():
    model.theta.copy_(torch.tensor([[0.], [0.1], [-0.1]]))
logits = model(X)
loss = student.data_loss(logits, y)
loss.backward() # 自动微分
p = student.sigmoid(logits.detach())
formula = X.T @ (p - y) / len(y) # 公式 X^T(p-y)/B
print(model.theta.grad.reshape(-1))
print(formula.reshape(-1))
```

预期输出与解释

两行输出相同, 均为

`tensor([-1.4901e-08, -2.3751e-01, 2.3751e-01])`

即约 $(0, -0.23751, 0.23751)$; 首项的 -1.49×10^{-8} 是浮点舍入误差, 理论值为 0。此时概率为 $(0.524979, 0.475021)$, 数据损失为 0.644397, 与前置讲义第 8 节的手算一致。若改用 `loss + 0.5 * model.penalty("l2")` 再反向传播, 梯度变为约 $(0, -0.18751, 0.18751)$: 两个特征分量分别加上 $\lambda\theta_1 = 0.05$ 与 $\lambda\theta_2 = -0.05$, 截距分量不变。这里取 $\lambda = 0.5$ 只是为了便于观察, 主实验使用 0.01。

公式与代码的常见错位

在 `forward` 中提前调用 `sigmoid`, 会使 `BCEWithLogitsLoss` 把概率再当作得分处理, 相当于计算 $\sigma(\sigma(z))$; 用 `theta` 代替 `theta[1:]` 会把截距也纳入惩罚; 把 `reduction` 改为 "sum" 会使梯度放大 B 倍, 相当于改变了学习率; 对 L1 惩罚, $|\theta_j|$ 在 0 处不可导, PyTorch 取该处的导数为 0, 因此普通梯度更新不保证权重恰好为零。

对照检查与参考答案

1. 自动微分得到的 `theta.grad` 与 $\frac{1}{B}\mathbf{X}_B^\top(\mathbf{p} - \mathbf{y}_B)$ 一致, 依赖于损失的哪一项设置?
 参考答案: 依赖 `reduction="mean"`, 它对应公式中的 $1/B$; 若用求和, 梯度为公式的 B 倍。
2. 为什么程序中看不到梯度公式, 却仍然实现了式中的更新?
 参考答案: `backward()` 按链式法则自动计算 ∇J_B , SGD 的 `step()` 执行 $\theta \leftarrow \theta - \eta \nabla J_B$, 二者合起来就是该更新式。
3. 损失曲线与训练总目标 J_B 为什么数值不同?
 参考答案: 曲线绘制的是不含惩罚项的数据损失 R ; 对带正则的方案, J_B 还包含 $\lambda\Omega$ 。

9.5 选择、评价与对应关系总表

模型选择与评价同样有明确的代码对应。`workflow.fit_candidate` 在每轮结束后计算验证集数据损失 $R(\theta)$, 保存其最小值所在轮次的参数; `train_candidates` 再从三种正则化方案中选择验证损失最低者并冻结。`final_test` 只在冻结后读取测试集一次。`binary_metrics` 由四种计数计算

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (9.5.1)$$

分母为零时代码约定返回 0。下表汇总本节的对应关系。

理论对象	数学表达	代码位置与形状
填补与标准化	$u_j^{(n)} = (\bar{x}_j^{(n)} - \mu_j)/s_j$	<code>fit_statistics</code> 估计 μ_j, s_j ; <code>apply_statistics</code> 应用
独热编码	每个类别一列 0/1 指示	<code>fit_categories</code> 、 <code>encode_categories</code> , 共 25 列
增广设计矩阵	$\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$, 首列为 1	<code>to_tensors</code> 返回 $\mathbf{X}$: (902, 29)
参数	$\theta \in \mathbb{R}^{d+1}$, θ_0 为截距	<code>self.theta</code> : (29, 1), <code>theta[0]</code> 为截距

理论对象	数学表达	代码位置与形状
线性得分	$\mathbf{z} = \mathbf{X}_B \boldsymbol{\theta}$	<code>forward: X @ self.theta, (B,1)</code>
概率	$\mathbf{p} = \sigma(\mathbf{z})$	<code>sigmoid(logits), (B,1)</code>
预测类别	$\hat{y} = \mathbf{1}[p \geq \tau]$	<code>probabilities >= threshold</code>
经验风险	$R_B = \frac{1}{B} \sum_n \mathcal{L}^{(n)}$	<code>data_loss: BCEWithLogitsLoss, 标量</code>
正则项	$\lambda \Omega$, 不含 θ_0	<code>strength * model.penalty(kind), 使用 theta[1:]</code>
总目标	$J_B = R_B + \lambda \Omega$	<code>objective</code>
梯度	$\nabla_{\boldsymbol{\theta}} J_B$	<code>objective.backward() 后的 model.theta.grad: (29,1)</code>
参数更新	$\boldsymbol{\theta}^{[t+1]} = \boldsymbol{\theta}^{[t]} - \eta \nabla J_B$	<code>optimizer.step(); 每批先 zero_grad()</code>
模型选择	验证集 R 最小	<code>fit_candidate、train_candidates</code>
评价指标	四种计数与四项指标	<code>binary_metrics; 测试由 final_test 执行一次</code>

10 实验报告要求与提交

10.1 报告结构

报告以 `report.pdf` 提交，即 PDF (Portable Document Format, 可移植文档格式) 文件，首页应注明实验名称、学号与姓名。报告参照模板按以下七个部分撰写：

- 1. **实验目的。**说明本实验需要理解和掌握的内容。
- 2. **实验环境与数据。**软件版本与运行设备；数据来源、划分、类别比例与缺失情况；被排除的字段与编码后的维数。
- 3. **实验原理。**简述模型、损失函数、正则化、优化方法与评价指标，篇幅建议不超过一页；可参照第 9 节的顺序组织，并写明各公式对应的代码位置。
- 4. **实验内容与步骤。**C0–C7 的完成情况；逐检查点给出对应函数、实现思路与关键代码、小例子的输入与预期/实际输出及分析。每处通常摘录 3–10 行关键代码，不粘贴整份文件；C2 须同时覆盖数值与类别处理，C6 须包含手算过程。
- 5. **实验结果。**训练设置、三种方案的验证结果表、测试前的选择理由、一张损失曲线、最终测试指标与混淆矩阵；选做对照单独列出验证结果。
- 6. **结果分析与讨论。**分析误报与漏判、与多数类基线的比较、至少两个验证阈值的权衡以及正则化是否改善表现；讨论同年分类、数据来源、非因果解释与跨年泛化等局限。
- 7. **实验总结。**概括主要结论、收获与建议。

报告采用客观、规范的书面语叙述；图、表应编号并配标题，正文须引用并解释。

10.2 各检查点的最低证据

阶段	至少一项可复核的证据
C0	原权重与修改权重后的矩阵乘法结果，并说明张量形状
C1	含缺失标签的三行样例，以及数值字符串的清洗结果
C2	均值、尺度与验证值的标准化结果；已知、缺失与未知类别的独热编码行
C3	第 3 条样本与标签的配对，以及五条样本得到的批次 2, 2, 1
C4	给定参数与输入下的得分、概率与张量维度
C5	零得分时的损失、两种惩罚值与截距的处理
C6	概率、梯度、一次更新的结果，以及手算与程序输出的对照
C7	四种计数的小例子；综合部分引用完整运行的实际结果

同一张综合图表在报告中只需出现一次，各检查点处可直接引用。报告中的全部数值须来自本人提交的代码；未完成的内容应如实标注，不得以教师参考数值代替本人的运行结果。

10.3 提交结构与评分建议

提交文件为 学号 _ 姓名 _lab03.zip，其中包含 report.pdf 与 code/ 目录。code/ 中应保留学生实现、运行入口、support、workflow、checkpoints、依赖文件，以及 data/classroom/ 下的三份数据与划分说明。使用 Notebook 时，需同时保留 logic.ipynb 与 student.py。

压缩包中无需包含运行环境、缓存、原始数据表、教师答案或课程讲义；输出图表应放入报告，模型权重与预测明细可自行保留。截止时间与提交平台以课程通知为准。

建议比例	核对内容
数据准备 30%	数值清洗、训练统计量、独热编码、未知类别与列顺序
模型实现 40%	参数与前向计算、概率接口、数据损失与两种惩罚
训练评价 30%	梯度更新、验证选择、最终测试，以及结果解释与反馈

上述比例是对原实验指导书评分结构的教学适配，如课程另有通知，以通知为准。选做对照用于拓展展示，不设必达准确率，也不按截图数量或篇幅评分。

10.4 提交前自查与常见问题

- 全部检查点均已通过，报告中的小例子可以复算；将压缩包解压到其他位置后，C0 可以直接运行。
- 两条运行命令使用相同的数据目录与输出目录；更换数据时应使用新的输出目录。
- 若出现维数不一致，应检查未知类别是否新增了列、拼接顺序是否改变、首列常数是否遗漏。
- 若损失异常，应检查数值缺失、特征尺度、是否重复调用 sigmoid、学习率大小，以及是否在反向传播前切断了计算图。
- 若第二次更新与手算不一致，应检查每个批次是否清除了梯度。
- 训练总目标与曲线数值不同属于正常现象：曲线是不含正则项的数据损失，而非带惩罚的总目标。
- 测试结果很高并不代表任务设置正确，应首先核对输入中是否混入了排名、总分或按排名排序的行号。

10.5 参考验证结果

在本环境、固定划分与上述设置下，教师参考实现的三种方案均在第 60 轮取得各自最低的验证损失：无正则约为 0.371792，L2 约为 0.379552，L1 约为 0.392744；三者的验证准确率均为 0.83。因此主实验选择无正则方案，这也说明正则化并不必然改善验证表现。若本人的验证结果与上述数值明显不同，应先检查预处理与训练设置是否一致。

本手册不提供测试集的参考结果。学生应在报告中记录冻结方案后运行一次最终测试，并以本人的测试结果为准；测试结果只用于最终评价，不用于与参考值对照后回调模型。

11 资料与改编说明

11.1 与原实验指导书的关系

本实验沿用原实验指导书第一章逻辑回归的概率、损失、正则化、训练与评价主线，采用 QS 大学数据，并加入数值缺失处理、独热编码、Unknown 类别与列对齐等内容。增广首列用于表示截距，前向计算返回得分，由 `BCEWithLogitsLoss` 接收得分计算损失。

本次数据共 1504 所学校、8 个原始输入字段，分层划分为 902/300/302。报告中的划分、类别比例与各项指标均应以本数据的实际运行结果为准。

11.2 实际引用资料

- 项目上级目录《人工智能数学原理与算法：实验指导书（最终）.pdf》第 1 章：第 1.4 节对应数据清洗，第 1.5 节对应模型与训练，第 1.6 节对应评分结构。
- 同级实验 01 的实验手册与报告模板：提交组织方式；实验 02 的代码：检查点风格。
- 前置讲义列出的课程总表、逻辑回归课件及数学记号说明。
- `code/data/README.md`、原始数据表与划分摘要：下载来源、字段映射、标签与真实缺失；`teacher/prepare_classroom_data.py`：可复现的课堂数据划分。
- [QS 官方学校分类说明](#)；[QS 排名方法说明](#)，核对日期为 2026-10-08。

教师参考实现与验收记录见 `teacher/README.md`，学生材料包中不含参考答案。

11.3 实验退出卡

实验结束时，请用一句话分别回答以下四个问题：编码表为什么也是需要拟合的状态？排名行号为什么不能输入模型？比较不同正则化方案时为什么要使用同一种验证损失？为什么一次划分上的良好表现并不意味着能够预测下一年的大学排名？